

Лабораторна робота №9

З дисципліни: Базы даних та інформаційні системи
Студента групи МІТ-31: Якоба Р.В.

Тема: Розширені можливості Redis

Мета роботи: Закріпити знання про роботу з Redis та ознайомитися з розширеним функціоналом — транзакціями, скриптами на Lua, публікацією/підпискою (Pub/Sub) та потоками Redis Streams

Завдання:

1. Транзакції у Redis

а. Ознайомтесь з командами MULTI, EXEC, DISCARD, WATCH.

```
WATCH balance
MULTI
DECR balance
INCR purchases
EXEC
```

б. Створіть просту транзакцію, що додає кілька ключів.

Приклад базової транзакції:

```
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> SET key1 "Hello"
QUEUED
127.0.0.1:6379(TX)> SET key2 "World"
QUEUED
127.0.0.1:6379(TX)> EXEC
1) OK
2) OK
```

с. Спробуйте використати WATCH для імітації конкурентного доступу.

Після EXEC обидві команди виконуються одночасно.

Імітація конкурентного доступу з WATCH

У CLI 1:

```
127.0.0.1:6379> SET balance 100
OK
127.0.0.1:6379> SET purchases 0
OK
127.0.0.1:6379> WATCH balance
OK
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> DECR balance
QUEUED
127.0.0.1:6379(TX)> INCR purchases
QUEUED
```

У CLI 2 (нове вікно):

```
127.0.0.1:6379> SET balance 999
OK
```

В CLI 1 вводимо і ось що бачимо:

```
127.0.0.1:6379(TX)> EXEC
(nil)
```

Redis відповість (nil) — транзакція не виконалась, бо balance змінився під час WATCH.

2. Lua-скрипти в Redis

- Ознайомтесь із командою EVAL.
<https://redis.io/docs/latest/commands/eval/>
- Напишіть простий Lua-скрипт, який перевіряє наявність ключа і створює його, якщо не існує.

Наприклад:

```
EVAL "if redis.call('exists', KEYS[1]) == 0 then
    return redis.call('set', KEYS[1], ARGV[1])
else
    return 'exists'
end"
1 testkey "value"
```

```
127.0.0.1:6379> EVAL "if redis.call('exists', KEYS[1]) == 0 then return redis.call('set', KEYS[1], ARGV[1]) else return 'exists' end" 1 testkey "value"
"exists"
```

Якщо запустити скрипт перший раз → створить testkey, якщо запустити ще раз → Redis скаже 'exists'. (на скриншоті 2 варіант)

3. Механізм Pub/Sub

- a. В одному терміналі запустіть підписку на канал:

SUBSCRIBE news

```
127.0.0.1:6379> SUBSCRIBE news
1) "subscribe"
2) "news"
3) (integer) 1
Reading messages... (press Ctrl-C to quit or any key to type command)
```

- b. В іншому терміналі виконайте публікацію повідомлення:

PUBLISH news "Redis is awesome!"

```
127.0.0.1:6379> PUBLISH news "Redis is awesome!"
(integer) 1
127.0.0.1:6379>
```

Перше вікно отримало повідомлення:

```
127.0.0.1:6379> SUBSCRIBE news
1) "subscribe"
2) "news"
3) (integer) 1
1) "message"
2) "news"
3) "Redis is awesome!"
Reading messages... (press Ctrl-C to quit or any key to type command)
```

Вивчіть можливість використання Pub/Sub для чатів або нотифікацій.

4. Redis Streams (поток даних)

- a. Додайте події до потоку:

XADD mystream * sensor-id 1234 temperature 19.8

```
127.0.0.1:6379> XADD mystream * sensor-id 1234 temperature 19.8
"1744286269113-0"
127.0.0.1:6379> XADD mystream * sensor-id 1235 temperature 20.1
"1744286269116-0"
127.0.0.1:6379> XRANGE mystream 0 +
```

- b. Прочитайте останні події:

XRANGE mystream - +

```
127.0.0.1:6379> XRANGE mystream - +
1) 1) "1744286269113-0"
  2) 1) "sensor-id"
    2) "1234"
    3) "temperature"
    4) "19.8"
2) 1) "1744286269116-0"
  2) 1) "sensor-id"
    2) "1235"
    3) "temperature"
    4) "20.1"
```

с. Створіть споживача потоку та зчитайте нові повідомлення:

XREAD COUNT 2 STREAMS mystream 0

```
127.0.0.1:6379> XREAD COUNT 2 STREAMS mystream 0
1) 1) "mystream"
  2) 1) 1) "1744286269113-0"
    2) 1) "sensor-id"
      2) "1234"
      3) "temperature"
      4) "19.8"
  2) 1) "1744286269116-0"
    2) 1) "sensor-id"
      2) "1235"
      3) "temperature"
      4) "20.1"
```

Після першого разу можна використовувати ID останнього повідомлення, щоб зчитувати тільки нові:

```
127.0.0.1:6379> XREAD STREAMS mystream 1712593331424-0
1) 1) "mystream"
  2) 1) 1) "1744286269113-0"
    2) 1) "sensor-id"
      2) "1234"
      3) "temperature"
      4) "19.8"
  2) 1) "1744286269116-0"
    2) 1) "sensor-id"
      2) "1235"
      3) "temperature"
      4) "20.1"
```

5. (Додатково) Напишіть скрипт (Python/JS), який реалізує логіку Pub/Sub або читає дані зі Stream.

```
import redis

# Підключення до Redis
r = redis.Redis(host='localhost', port=6379, decode_responses=True)

# Починаємо читати з самого початку (0), або з '$' – тільки нові
stream_id = '0'

print("Читання з потоку 'mystream'... натисни Ctrl+C щоб зупинити")

try:
    while True:
        # Зчитати максимум 2 повідомлення
        messages = r.xread(['mystream': stream_id], count=2, block=5000) # 5 сек очікування

        for stream_name, events in messages:
            for event_id, data in events:
                print(f"\n[ID: {event_id}]")
                for key, value in data.items():
                    print(f"({key}): {value}")
                stream_id = event_id # оновлюємо останній ID
except KeyboardInterrupt:
    print("\nЗчитування завершено вручну.")
```

[1] 33.9s Python

... Читання з потоку 'mystream'... натисни Ctrl+C щоб зупинити

[ID: 1744286269113-0]
sensor-id: 1234
temperature: 19.8

[ID: 1744286269116-0]
sensor-id: 1235
temperature: 20.1

[ID: 1744286581850-0]
sensor-id: 1236
temperature: 21.3

```
127.0.0.1:6379> XADD mystream * sensor-id 1236 temperature 21.3
"1744286581850-0"
127.0.0.1:6379> XADD mystream * sensor-id 1238 temperature 31.3
"1744286654065-0"
127.0.0.1:6379> 
```

```
import redis

# Підключення до Redis
r = redis.Redis(host='localhost', port=6379, decode_responses=True)

# Починаємо читати з самого початку (0), або з '$' – тільки нові
stream_id = '0'

print("Читання з потоку 'mystream'... натисни Ctrl+C щоб зупинити")

try:
    while True:
        # Зчитати максимум 2 повідомлення
        messages = r.xread(['mystream': stream_id], count=2, block=5000) # 5 сек очікування

        for stream_name, events in messages:
            for event_id, data in events:
                print(f"\n[ID: {event_id}]")
                for key, value in data.items():
                    print(f"({key}): {value}")
                stream_id = event_id # оновлюємо останній ID
except KeyboardInterrupt:
    print("\nЗчитування завершено вручну.")
```

[1] 1m 43.8s Python

... Читання з потоку 'mystream'... натисни Ctrl+C щоб зупинити

[ID: 1744286269113-0]
sensor-id: 1234
temperature: 19.8

[ID: 1744286269116-0]
sensor-id: 1235
temperature: 20.1

[ID: 1744286581850-0]
sensor-id: 1236
temperature: 21.3

[ID: 1744286654065-0]
sensor-id: 1238
temperature: 31.3

Висновок:

Під час лабораторної роботи було розглянуто розширені можливості Redis, зокрема транзакції, скрипти на мові Lua, механізм публікації/підписки (Pub/Sub) та Redis Streams. Була продемонстрована робота з командами MULTI, EXEC, WATCH для транзакцій, створено простий Lua-скрипт, протестовано передачу повідомлень через канали, а також додавання та зчитування подій у потоках.